



WebX

2-Mark Questions

Module IV — MongoDB & REST API

1. What is MongoDB? How is it different from a relational database?
2. List any four data types supported by MongoDB.
3. What is Mongoose? Why is it used in Node.js applications?
4. Define REST API. What does REST stand for?
5. List the four CRUD operations and their corresponding HTTP methods in REST API.

Module V — Flask

1. What is Flask? Why is it called a micro-framework?
2. What is app routing in Flask? Give one example.
3. What is URL building in Flask? Which function is used for it?
4. What are Flask cookies? How are they set and retrieved?
5. What is the use of `@app.route()` decorator in Flask?

Module VI — Rich Internet Applications

1. What is AJAX? Expand the abbreviation and state its purpose.
2. What is an XMLHttpRequest object? What is its role in AJAX?
3. What is the difference between synchronous and asynchronous communication in AJAX?
4. What is a CMS? Give two examples of open source CMS used for RIA.

5. List any two applications of AJAX in blogs, wikis, or RSS feeds.

5-Mark Questions

Module IV — MongoDB & REST API

1. Explain the steps to connect MongoDB from Node.js with a suitable example.
2. What is Mongoose? Explain how it is used for structured schema and validation with an example.
3. Explain the rules of REST APIs. How are CRUD functions handled using HTTP methods?
4. Compare MongoDB with SQL databases on the basis of schema, scalability, query language, and use cases.

Module V — Flask

1. Explain Flask app routing and URL building with suitable examples.
2. What are Flask HTTP methods? Explain GET and POST methods with an example.
3. Explain file uploading in Flask with a complete example including HTML form and Flask code.

Module VI — Rich Internet Applications

1. What is AJAX? Explain how AJAX works with a neat diagram showing the AJAX request-response cycle.
2. Explain the key components used in developing RIA using AJAX — CSS, HTML, DOM, XMLHttpRequest, and JavaScript.
3. How is AJAX used as a REST client? Explain with a suitable example using XMLHttpRequest.

ANSWERS For 2-Mark Questions: Module IV — MongoDB & REST API

Q1. What is MongoDB? How is it different from a relational database?

MongoDB

- A NoSQL database that stores data in **JSON-like documents**.
- Schema-less, flexible, and designed for handling **unstructured or semi-structured data**.

Difference from Relational Database

- Relational DB uses **tables, rows, and columns**; MongoDB uses **collections and documents**.
- Relational DB requires a **fixed schema**; MongoDB allows **dynamic schema**.
- Relational DB is best for **structured data**; MongoDB is suited for **large, varied datasets**.

Q2. List any four data types supported by MongoDB.

- **String** – Stores text data.
- **Integer** – Stores numeric values.
- **Boolean** – Stores true/false values.
- **Array** – Stores multiple values in a single field.

Q3. What is Mongoose? Why is it used in Node.js applications?

Mongoose

- An **Object Data Modeling (ODM) library** for MongoDB in Node.js.
- Provides a way to define **schemas and models** for MongoDB collections.

Why Used in Node.js

- Simplifies **data validation and querying**.
- Adds structure to MongoDB's **schema-less documents**.

- Makes database operations more **organized and consistent**.

Q4. Define REST API. What does REST stand for?

REST API

- A set of rules for building web services using **HTTP methods** (GET, POST, PUT, DELETE).
- Enables communication between client and server in a **stateless** manner.

REST stands for

- **Representational State Transfer**.

Q5. List the four CRUD operations and their corresponding HTTP methods in REST API.

CRUD Operations with HTTP Methods

- **Create** → **POST**
- **Read** → **GET**
- **Update** → **PUT**
- **Delete** → **DELETE**

ANSWERS For 2-Mark Questions: Module V — Flask

Q1. What is Flask? Why is it called a micro-framework?

Flask

- A lightweight **Python web framework** used to build web applications.
- Provides essential tools for routing, request handling, and templates.

Why Micro-framework

- Minimal features compared to full frameworks like Django.
- Focuses only on the **core functionality** (no built-in ORM, authentication, etc.).
- Developers can add extensions as needed, keeping it **simple and flexible**.

Q2. What is app routing in Flask? Give one example.

App Routing in Flask

- App routing means **mapping URLs to specific functions** in a Flask application.
- It helps define what code should run when a user visits a particular URL.

Example

```
@app.route('/')
def home():
    return "Welcome to Flask!"
```

Q3. What is URL building in Flask? Which function is used for it?

URL Building in Flask

- It means **generating URLs dynamically** for functions instead of hardcoding them.
- Ensures links remain valid even if route definitions change.

Function Used

- The function is `url_for()`.

Example:

```
url_for('home')
```

Q4. What are Flask cookies? How are they set and retrieved?

Flask Cookies

- Cookies are small pieces of data stored on the **client's browser**.
- In Flask, they are used to **remember user information** across requests.

Set and Retrieve

- **Set:** `res.set_cookie('username', 'TheHonoredOne')`
- **Retrieve:** `request.cookies.get('username')`

Q5. What is the use of @app.route() decorator in Flask?

Use of `@app.route()` in Flask

- It is a **decorator** used to bind a URL to a specific function.
- Defines which function should run when a user visits a given URL.

Example:

```
@app.route('/about')
def about():
    return "About Page"
```

ANSWERS For 2-Mark Questions: Module VI — Rich Internet Applications

Q1. What is AJAX? Expand the abbreviation and state its purpose.

- Stands for **Asynchronous JavaScript and XML**.
- Technique used to **send and receive data asynchronously** from a server without reloading the entire web page.
- Purpose: Improves **user experience** by making web applications faster and more interactive.

Q2. What is an XMLHttpRequest object? What is its role in AJAX?

XMLHttpRequest Object

- A built-in **JavaScript object** used to interact with servers.
- Core tool for implementing **AJAX requests**.

Role in AJAX

- Sends requests to the server **asynchronously**.
- Retrieves data (like JSON, XML, or text) without reloading the page.
- Enables **dynamic updates** to web content for better user experience.

Q3. What is the difference between synchronous and asynchronous communication in AJAX?

Synchronous Communication

- The browser **waits** for the server response before continuing.
- Page becomes **blocked** until the request completes.

Asynchronous Communication

- Browser does **not wait**; other tasks continue while the request is processed.
- Allows **non-blocking updates**, improving speed and user experience.

Q4. What is a CMS? Give two examples of open source CMS used for RIA.

CMS

- Stands for **Content Management System**.
- Software used to **create, manage, and publish digital content** easily without deep coding knowledge.
- Often used for building **Rich Internet Applications (RIA)**.

Examples of Open Source CMS

- **WordPress**
- **Drupal**

Q5. List any two applications of AJAX in blogs, wikis, or RSS feeds.

Applications of AJAX in Blogs, Wikis, or RSS Feeds

- **Auto-refreshing comments or posts** without reloading the entire page.
- **Dynamic content loading** such as fetching new RSS feed entries or wiki updates in real time.
- **Live search/autocomplete in blogs or wikis** – AJAX can fetch search results instantly as the user types, without reloading the page.

ANSWERS For 5-Mark Questions: Module IV — MongoDB & REST API

Q1. Explain the steps to connect MongoDB from Node.js with a suitable example.

Steps to Connect MongoDB from Node.js

1. Install MongoDB Driver

- Use npm to install the official MongoDB package:

```
npm install mongodb
```

2. Import MongoDB Client

- Require the MongoDB module in your Node.js file:

```
const { MongoClient } = require('mongodb');
```

3. Define Connection URL & Database Name

- Provide the MongoDB connection string (local or cloud):

```
const url = "mongodb://localhost:27017";  
const dbName = "myDatabase";
```

4. Connect to MongoDB Server

- Use `MongoClient.connect()` to establish connection:

```
MongoClient.connect(url, { useUnifiedTopology: true }, (err, client) => {  
  if (err) throw err;  
  console.log("Connected successfully to MongoDB!");  
  const db = client.db(dbName);  
});
```

5. Perform Operations (CRUD)

- Example: Insert a document into a collection:

```
db.collection("students").insertOne({ name: "Yuji Itadori", course: "Jujutsu Sorcerer" }, (err, res) => {  
  if (err) throw err;  
  console.log("Document inserted");  
  client.close(); // Close connection after operation  
});
```

Q2. What is Mongoose? Explain how it is used for structured schema and validation with an example.

Mongoose

- Mongoose is an **Object Data Modeling (ODM)** library for MongoDB and Node.js.

- It provides a way to define **structured schemas** for collections and supports **validation rules** to ensure data integrity.
- Unlike raw MongoDB driver, Mongoose adds a **layer of abstraction** that makes handling data more organized and predictable.

How Mongoose Uses Structured Schema & Validation

1. Schema Definition

- You define a schema that specifies the structure of documents (fields, types, constraints).
- Example: `name` must be a string, `age` must be a number.

2. Validation Rules

- Schemas can enforce rules like `required`, `unique`, `minlength`, `maxlength`, or custom validators.
- This ensures only valid data is stored in the database.

3. Model Creation

- A model is created from the schema, representing a MongoDB collection.
- Models allow CRUD operations while automatically applying schema rules.

4. Data Operations

- When inserting or updating data, Mongoose checks against the schema and rejects invalid entries.

Example: Structured Schema with Validation:

```
const mongoose = require('mongoose');

// Connect to MongoDB
mongoose.connect('mongodb://localhost:27017/myDatabase', { use
  eUrlParser: true, useUnifiedTopology: true });

// Define Schema with validation
const studentSchema = new mongoose.Schema({
  name: { type: String, required: true },           // must be pr
```

```

provided
  age: { type: Number, min: 18 },           // must be >=
18
  email: { type: String, unique: true }     // must be un
ique
});

// Create Model
const Student = mongoose.model('Student', studentSchema);

// Insert Document
const newStudent = new Student({ name: "Kinji Hakari", age: 1
9, email: "hakari@gambling.com" });

newStudent.save()
  .then(() => console.log("Student saved successfully"))
  .catch(err => console.log("Validation Error:", err.messag
e));

```

Q3. Explain the rules of REST APIs. How are CRUD functions handled using HTTP methods?

Rules of REST APIs

REST (Representational State Transfer) is an architectural style for designing web services. The key rules are:

1. **Statelessness** – Each request from client to server must contain all necessary information; the server does not store client state.
2. **Client-Server Separation** – The client handles the user interface, while the server manages data and logic.
3. **Uniform Interface** – Resources are accessed using standard methods (HTTP verbs) and identified by unique URIs.

4. **Resource Representation** – Data is represented in formats like JSON or XML, making it easy to transfer.
5. **Cacheable Responses** – Responses should define whether they can be cached to improve performance.

CRUD Functions with HTTP Methods

- **Create → POST**
 - Used to add new resources.
 - Example: `POST /users` creates a new user.
- **Read → GET**
 - Retrieves data from the server.
 - Example: `GET /users/1` fetches details of user with ID 1.
- **Update → PUT/PATCH**
 - `PUT` replaces an entire resource, while `PATCH` updates part of it.
 - Example: `PUT /users/1` updates user details.
- **Delete → DELETE**
 - Removes a resource from the server.
 - Example: `DELETE /users/1` deletes user with ID 1.

Q4. Compare MongoDB with SQL databases on the basis of schema, scalability, query language, and use cases.

Aspect	MongoDB (NoSQL)	SQL Databases (Relational)
Schema	Schema-less; documents in a collection can have different fields and structures.	Fixed schema; tables require predefined columns and data types.
Scalability	Horizontally scalable (sharding across multiple servers).	Primarily vertically scalable (adding more power to a single server).
Query Language	Uses MongoDB Query Language (MQL), JSON-like syntax.	Uses Structured Query Language (SQL) with joins and relational

Aspect	MongoDB (NoSQL)	SQL Databases (Relational)
		operations.
Use Cases	Best for big data, real-time analytics, IoT, content management, flexible apps.	Best for financial systems, ERP, CRM, and applications needing strict ACID.

ANSWERS For 5-Mark Questions: Module V — Flask

Q1. Explain Flask app routing and URL building with suitable examples.

Flask App Routing

- Routing in Flask means **mapping URLs to specific functions**.
- The `@app.route()` decorator is used to define which function should run when a user visits a particular URL.
- It helps organize the application by linking web addresses to Python functions.

Example of Routing

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return "Welcome to Home Page"

@app.route('/about')
def about():
    return "About Page"
```

Flask URL Building

- Instead of hardcoding URLs, Flask provides the `url_for()` function to dynamically generate URLs.
- This ensures that if routes change, links remain valid automatically.

Example of URL Building

```
from flask import Flask, url_for

app = Flask(__name__)

@app.route('/profile/<username>')
def profile(username):
    return f"Profile page of {username}"

@app.route('/test')
def test():
    return url_for('profile', username='GojoKitkat')
```

Q2. What are Flask HTTP methods? Explain GET and POST methods with an example.

Flask HTTP Methods

- Flask supports different **HTTP methods** (GET, POST, PUT, DELETE, etc.) to handle client-server communication.
- By default, routes respond to **GET requests**, but you can explicitly allow other methods using the `methods` parameter in `@app.route()`.
- The two most commonly used methods are **GET** and **POST**.

GET Method

- Used to **retrieve data** from the server.
- Parameters are passed in the **URL query string**.
- Example:

```

from flask import Flask, request
app = Flask(__name__)

@app.route('/get_example', methods=['GET'])
def get_example():
    name = request.args.get('name')    # URL: /get_example?name=tanjiro
    return f"Hello {name}"

```

POST Method

- Used to **send data to the server** (e.g., form submission).
- Parameters are sent in the **request body**, not visible in the URL.
- Example:

```

@app.route('/post_example', methods=['POST'])
def post_example():
    name = request.form['name']    # Data sent via form
    return f"Welcome {name}"

```

Q3. Explain file uploading in Flask with a complete example including HTML form and Flask code.

File Uploading in Flask

- Flask allows users to upload files (images, documents, etc.) from an HTML form to the server.
- It uses the `request.files` object to access uploaded files.
- The uploaded file can be saved using the `.save()` method.

Steps for File Uploading

1. **Create HTML Form** with `enctype="multipart/form-data"` and `method="POST"`.

2. **Configure Flask App** to accept file uploads.
3. **Access File** using `request.files['file']`.
4. **Save File** to a desired directory.

Complete Example

HTML Form (templates/upload.html):

```
<!DOCTYPE html>
<html>
<head>
    <title>File Upload</title>
</head>
<body>
    <h2>Upload a File</h2>
    <form method="POST" enctype="multipart/form-data" action
    ="/upload">
        <input type="file" name="file">
        <input type="submit" value="Upload">
    </form>
</body>
</html>
```

Flask Code (app.py):

```
from flask import Flask, request, render_template
import os

app = Flask(__name__)

# Folder to save uploaded files
UPLOAD_FOLDER = 'uploads'
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

@app.route('/')
def index():
    return render_template("upload.html")
```



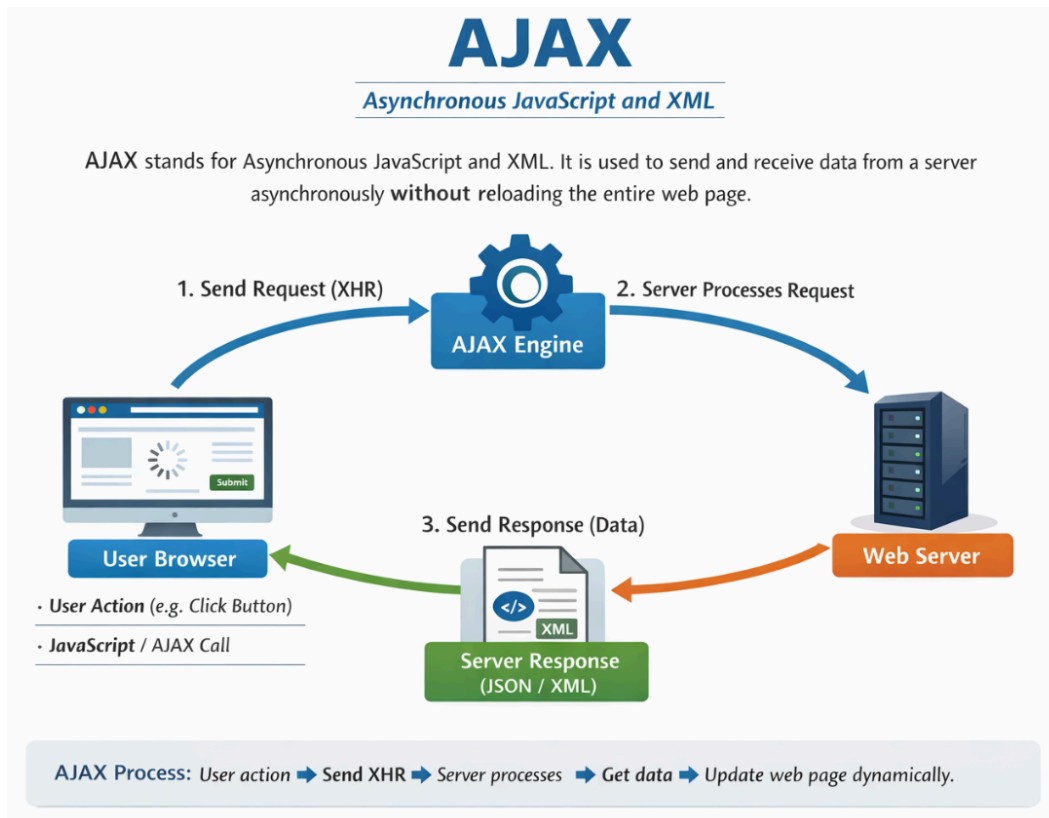
```
def index():
    return render_template('upload.html')

@app.route('/upload', methods=['POST'])
def upload_file():
    if 'file' not in request.files:
        return "No file part"
    file = request.files['file']
    if file.filename == '':
        return "No selected file"
    file.save(os.path.join(app.config['UPLOAD_FOLDER'], file.
filename))
    return f"File {file.filename} uploaded successfully!"

if __name__ == '__main__':
    app.run(debug=True)
```

ANSWERS For 5-Mark Questions: Module VI — Rich Internet Applications

Q1. What is AJAX? Explain how AJAX works with a neat diagram showing the AJAX request-response cycle.



AJAX (Asynchronous JavaScript and XML) is a technique that allows web pages to communicate with the server asynchronously, meaning data can be sent and received without reloading the entire page. It works by using the `XMLHttpRequest` object to send requests and update only parts of the page dynamically.

How AJAX Works (Request-Response Cycle)

1. User Action

- A user triggers an event (e.g., clicking a button, submitting a form).

2. AJAX Call (JavaScript)

- JavaScript creates an `XMLHttpRequest` object and sends a request to the server asynchronously.

3. Server Processing

- The server receives the request, processes it (e.g., fetch data from a database), and prepares a response.

4. Server Response

- The server sends back data in formats like JSON, XML, or plain text.

5. Update Web Page

- JavaScript updates only the relevant part of the page using DOM manipulation, without refreshing the entire page.

Example Code (AJAX in Action)

HTML + JavaScript

```
<!DOCTYPE html>
<html>
<head>
  <title>AJAX Example</title>
</head>
<body>
  <h2>AJAX Demo</h2>
  <button onclick="loadData()">Get Data</button>
  <div id="result"></div>

  <script>
    function loadData() {
      var xhr = new XMLHttpRequest();
      xhr.open("GET", "/data", true); // asynchronous request
      xhr.onload = function() {
        if (xhr.status == 200) {
          document.getElementById("result").innerHTML = xhr.responseText;
        }
      };
      xhr.send();
    }
  </script>
```

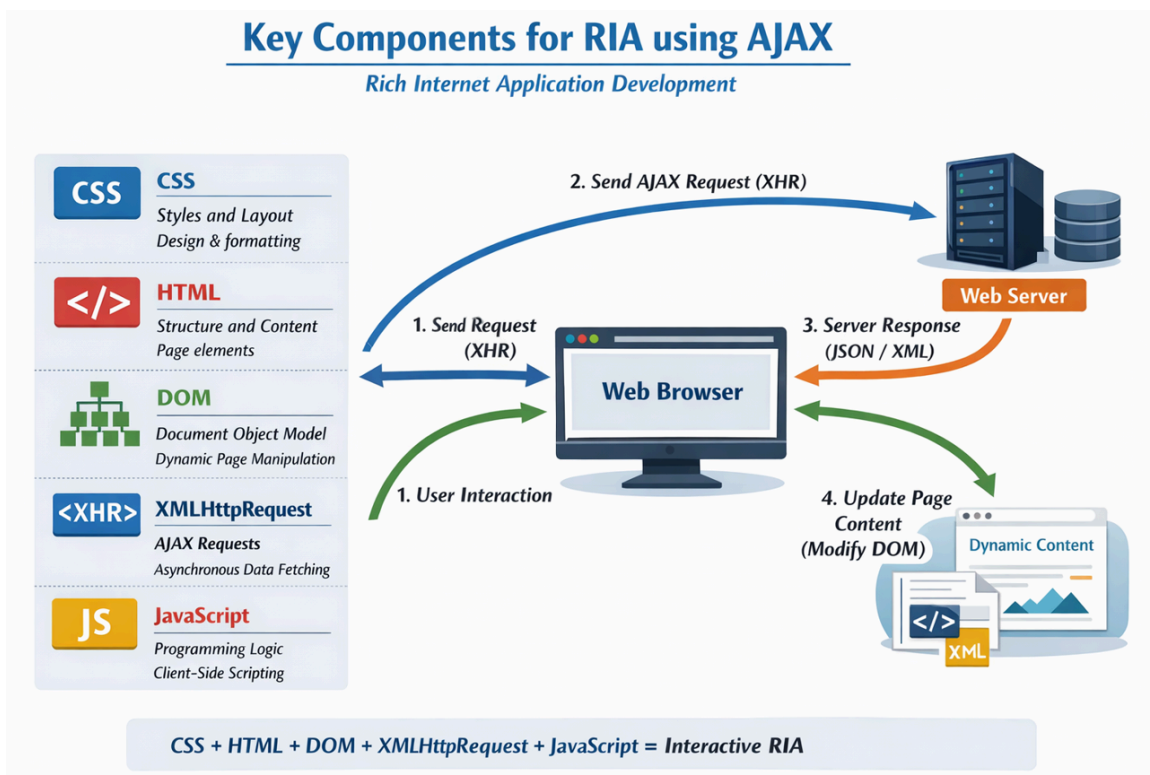
```
</body>
</html>
```

Flask Backend Example:

```
from flask import Flask
app = Flask(__name__)

@app.route('/data')
def data():
    return "Hello people, this is AJAX response!"
```

Q2. Explain the key components used in developing RIA using AJAX — CSS, HTML, DOM, XMLHttpRequest, and JavaScript.



1. HTML (HyperText Markup Language)

- Defines the **structure and content** of a web page.
- Provides elements like forms, buttons, and divs that interact with AJAX.
- Example: A form or button triggers an AJAX call when clicked.

2. CSS (Cascading Style Sheets)

- Controls the **presentation and layout** of the web page.
- Ensures that dynamically updated content (via AJAX) maintains consistent styling.
- Example: Styling a section that updates asynchronously without page reload.

3. DOM (Document Object Model)

- Represents the **structure of the HTML document** as a tree of objects.
- AJAX uses JavaScript to **manipulate the DOM** dynamically—adding, removing, or updating elements based on server responses.
- Example: Updating a `<div>` with new data fetched via AJAX.

4. XMLHttpRequest Object

- Core component that enables **asynchronous communication** between browser and server.
- Sends requests and receives responses without reloading the page.
- Example: `xhr.open("GET", "data.json", true)` initiates an AJAX request.

5. JavaScript

- Acts as the **controller** that ties all components together.
- Handles user events, creates AJAX requests, processes server responses, and updates the DOM.
- Example: JavaScript function triggers `XMLHttpRequest` and updates the page dynamically.

Q3. How is AJAX used as a REST client? Explain with a suitable example using XMLHttpRequest.

AJAX as a REST Client

- AJAX can act as a **REST client** by using the `XMLHttpRequest` object to send HTTP requests (GET, POST, PUT, DELETE) to RESTful APIs.
- It communicates with the server asynchronously, allowing data exchange without reloading the page.
- Each HTTP method corresponds to a **CRUD operation**:
 - **GET** → Read data
 - **POST** → Create data
 - **PUT/PATCH** → Update data
 - **DELETE** → Remove data

Working Process

1. JavaScript creates an `XMLHttpRequest` object.
2. Opens a connection to the REST API endpoint using the desired HTTP method.
3. Sends the request asynchronously.
4. Receives and processes the server's JSON or XML response.
5. Updates the web page dynamically using DOM manipulation.

Example: Using AJAX as REST Client (GET Request)

```
<!DOCTYPE html>
<html>
<head>
  <title>AJAX REST Client Example</title>
</head>
<body>
  <h2>Fetch User Data</h2>
  <button onclick="getUser()">Get User</button>
  <div id="result"></div>
```

```

<script>
  function getUser() {
    var xhr = new XMLHttpRequest();
    xhr.open("GET", "https://jsonplaceholder.typicode.com/users/1", true); // REST API endpoint
    xhr.onload = function() {
      if (xhr.status === 200) {
        var user = JSON.parse(xhr.responseText);
        document.getElementById("result").innerHTML =
          "Name: " + user.name + "<br>Email: " + user.email;
      }
    };
    xhr.send();
  }
</script>
</body>
</html>

```